

Frontend Developer — Step 4: React

Expanded notes: definitions, examples, hooks, state, forms, APIs, routing, context, debugging and project checklist

1. React Fundamentals

Definition

React is a JavaScript library for building user interfaces using reusable components. It lets developers describe UI from current data and state.

Core terms

Component = reusable UI unit. JSX = JavaScript syntax extension that looks like HTML. Props = inputs passed from parent to child. State = changing data owned by a component. Render = React calculates UI for the current state/props.

Why React?

Component reuse, predictable UI updates, composition, ecosystem and developer tooling.

```
function Welcome({ name }) {  
  return <h1>Hello, {name}</h1>;  
}  
  
function App() {  
  return <Welcome name="Manthan" />;  
}
```

JSX is not HTML. It is transformed into JavaScript. Common differences include className instead of class and camelCase event names.

2. Components, JSX & Props

Component definition

A component represents a reusable part of an interface. Keep components focused and compose small components into larger screens.

```
function Button({ label, onClick }) {  
  return <button onClick={onClick}>{label}</button>;  
}
```

Props definition

Props are read-only inputs supplied by a parent. A child should not mutate its props.

```
function Card({ title, description }) {  
  return <article><h2>{title}</h2><p>{description}</p></article>;  
}  
  
<Card title="React" description="UI library" />
```

Composition

Build complex interfaces from Header, Card, Form, List, Sidebar and Footer components.

3. State, Events & Forms

State definition

State is component data that persists between renders and whose update can cause the UI to render again.

```
import { useState } from "react";
```

```
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}
```

Immutability

Do not directly mutate existing state. Create a new object/array and pass it to the setter.

```
setUser(prev => ({ ...prev, name: "Asha" }));
setItems(prev => [...prev, newItem]);
```

Controlled forms

A controlled input gets its value from React state and updates that state through onChange.

```
const [email, setEmail] = useState("");
<input value={email} onChange={e => setEmail(e.target.value)} />
```

4. Lists, Keys & Conditional Rendering

Lists

Use Array.map() to convert data into UI elements.

```
const skills = ["HTML", "CSS", "JavaScript"];
<ul>{skills.map(s => <li key={s}>{s}</li>)}</ul>
```

Key definition

A key gives list items stable identity so React can track additions, removals and changes. Prefer stable unique IDs.

Conditional rendering

Use if/early return, ternary or && to show loading, error, empty and success states.

```
return loading ? <p>Loading...</p> : <CareerList />;
```

5. Hooks & useEffect

Hook definition

Hooks let functional components use state and other React features.

Rules of Hooks

Call hooks only at the top level of React components or custom hooks. Do not call them inside loops, conditions or nested functions.

useEffect definition

useEffect synchronizes a component with an external system such as a network request, timer, subscription or browser API.

```
useEffect(() => {
  document.title = `Count: ${count}`;
}, [count]);
```

Do not use useEffect for every calculation. If a value can be calculated during rendering, normal JavaScript is usually simpler.

6. API Integration

API flow

UI action/mount → HTTP request → loading → response → JSON parsing → state update → UI. Handle error and empty states.

```
async function loadCareers() {
  const res = await fetch("/api/careers");
  if (!res.ok) throw new Error("Request failed");
  return await res.json();
}
```

POST example

```
await fetch("/api/careers", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(data)
});
```

7. State Management & Context

Lifting state up

When multiple components need the same state, place it in their nearest common parent and pass data/handlers down.

Context definition

Context provides shared values to descendants without passing props through every intermediate component. Theme, locale and authentication are common examples.

External state libraries

Large applications may use dedicated state-management libraries. Learn local state, lifting state and Context first.

8. Routing

Definition

Client-side routing maps URLs to views in a single-page application.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/careers" element={<Careers />} />
  <Route path="/careers/:id" element={<CareerDetails />} />
</Routes>
```

Dynamic parameter

```
const { id } = useParams();
```

Good practice

Use meaningful URLs, not-found handling, parameter validation and useful loading/error states.

9. Performance & Debugging

Performance

First make the application correct and readable, then measure. Lazy loading, code splitting and memoization can help when profiling shows a real need.

Debugging

Reproduce → Console → Network → inspect props/state → isolate component/API → fix → retest.

Interview checklist

Components, JSX, props, state, events, keys, hooks, effects, forms, routing, API integration, Context, error handling and basic performance.

10. React Project Checklist

Project

Build a Career Explorer with career list, search/filter, detail page, roadmap, resources, routing, API integration, loading/error states and responsive UI.

Checklist

Components • JSX • Props • State • Events • Forms • Lists/keys • Hooks • Effects • API • Routing • Context • Debugging • Responsive UI • Deployment.